Flash storage and filesystems



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



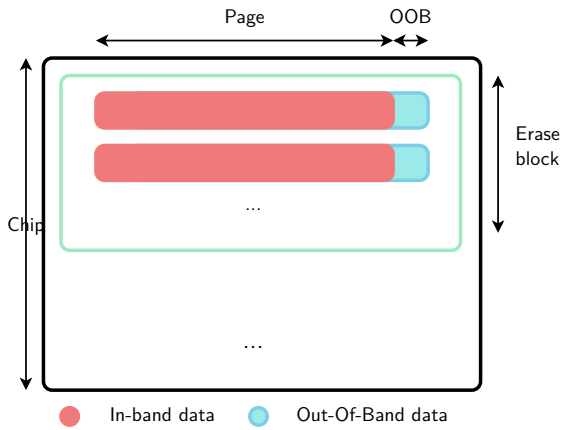
Block devices vs raw flash devices: reminder

- Block devices:
  - Allow for random data access using fixed size blocks
  - Do not require special care when writing on the media
  - Block size is relatively small (minimum 512 bytes, can be increased for performance reasons)
  - Considered as reliable (if the storage media is not, some hardware or software parts are supposed to make it reliable)
- Raw flash devices:
  - Flash chips directly driven by the flash controller on your SoC. You can control how they are managed.
  - Allow for random data access too, but require erasing before writing on the media.
  - Read and write (for example 4 KiB) don't use the same block size as erasing (for example 128 KiB).
  - Multiple flash technologies: NOR flash, NAND flash (Single Level Cell - SLC: 1 bit per cell, MLC: multiple bits per cell).



NAND flash storage: constraints

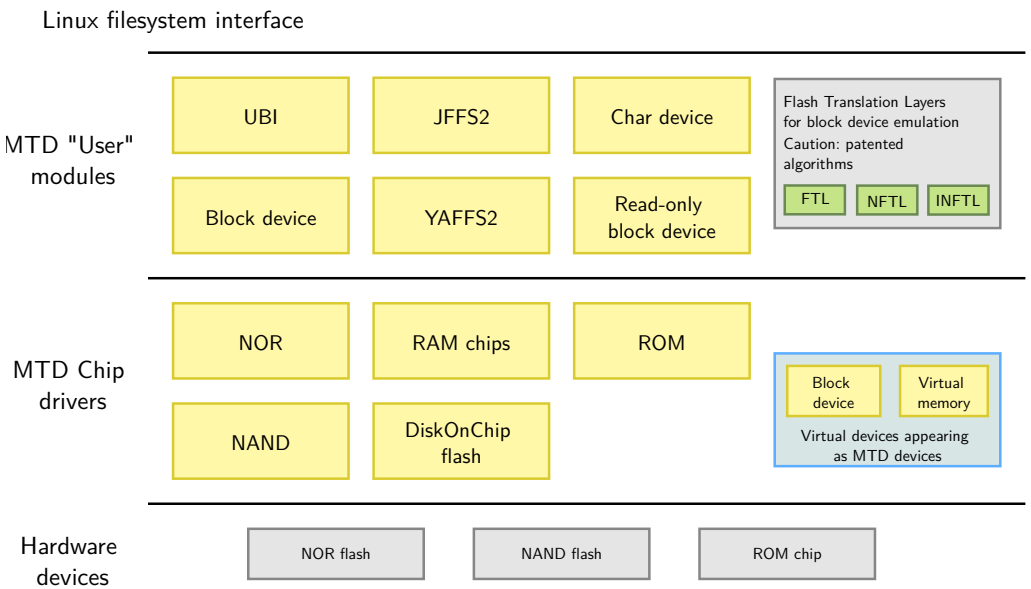
- Reliability
  - Reliability depends on flash technology (SLC, MLC)
  - Require mechanisms to recover from bit flips: ECC (Error Correcting Code), stored in the OOB (Out-Of-Band area)
- Lifetime
  - Relatively short lifetime: between 1,000,000 (SLC) and 1,000 (MLC) erase cycles per block
  - Wear leveling required to erase blocks evenly
  - Bad block detection/handling required too
- Widely used anyway in embedded systems for several reasons: low cost, high capacity, good read and write performance.





The MTD subsystem

- MTD stands for *Memory Technology Devices*
- Generic subsystem in Linux dealing with all types of storage media that are not fitting in the block subsystem
- Supported media types: RAM, ROM, NOR flash, NAND flash, Dataflash...
- Independent of the communication interface (drivers available for parallel, SPI, direct memory mapping, ...)
- Abstract storage media characteristics and provide a simple API to access MTD devices





MTD partitioning

- MTD devices are usually partitioned
  - It allows to use different areas of the flash for different purposes: read-only filesystem, read-write filesystem, backup areas, bootloader area, kernel area, etc.
- Unlike block devices, which contains their own partition table, the partitioning of MTD devices is described externally (don't want to put it in a flash sector which could become bad)
  - Specified in the board Device Tree (default partitions, not always relevant)
  - Specified through the kernel command line
- MTD partitions are defined through the `mtdparts` parameter in the kernel command line
- U-Boot understands the Linux syntax via the `mtdparts` and `mtdids` variables



MTD partitions on Linux

- Each partition becomes a separate MTD device
- Different from block device labeling (`sda3`, `mmcblk0p2`)
- `/dev/mtd0` is the first enumerated partition on the system
- `/dev/mtd1` is the second enumerated partition on the system (either from a single flash chip or from a different one).
- Note that the master MTD device (the device those partitions belong to) is not exposed in `/dev`



Commands to manage NAND devices

- From U-Boot
  - `help nand` to see all `nand` subcommands
  - `nand info`, `nand read`, `nand write`, `nand erase`…
- From Linux
  - **mtdchar** driver: one `/dev/mtdX` and `/dev/mtdXro` character device per partition.
  - Accessed through `ioctl()` operations to erase and flash the storage.
  - Used by these utilities: `flash_eraseall`, `nandwrite` Provided by the *mtd-utils* package, also available in BusyBox
  - There are also host commands in *mtd-utils*: `mkfs.jffs2`, `mkfs.ubifs`, `ubinize`…



Flash wear leveling

- Wear leveling consists in distributing erases over the whole flash device to avoid quickly reaching the maximum number of erase cycles on blocks that are written really often
- Can be done in:
  - the filesystem layer (JFFS2, YAFFS2, ...)
  - an intermediate layer dedicated to wear leveling (UBI)
- The wear leveling implementation is what makes your flash lifetime good or not



Flash file-systems

- `Standard` file systems (*ext2*, *ext4*...) are meant to work on block devices
- Specific file systems have been developed to deal with flash constraints
- These file systems are relying on the MTD layer to access flash chips
- There are several legacy flash filesystems which might be useful for small partitions: JFFS2, YAFFS2.
- Nowadays, UBI/UBIFS is the de facto standard for medium to large capacity NANDs

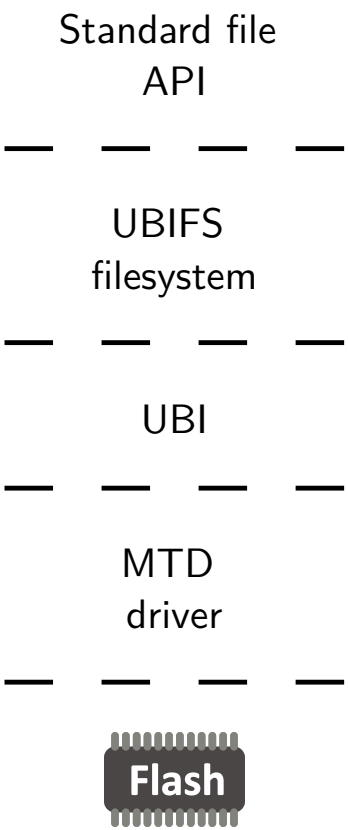


UBI (1)

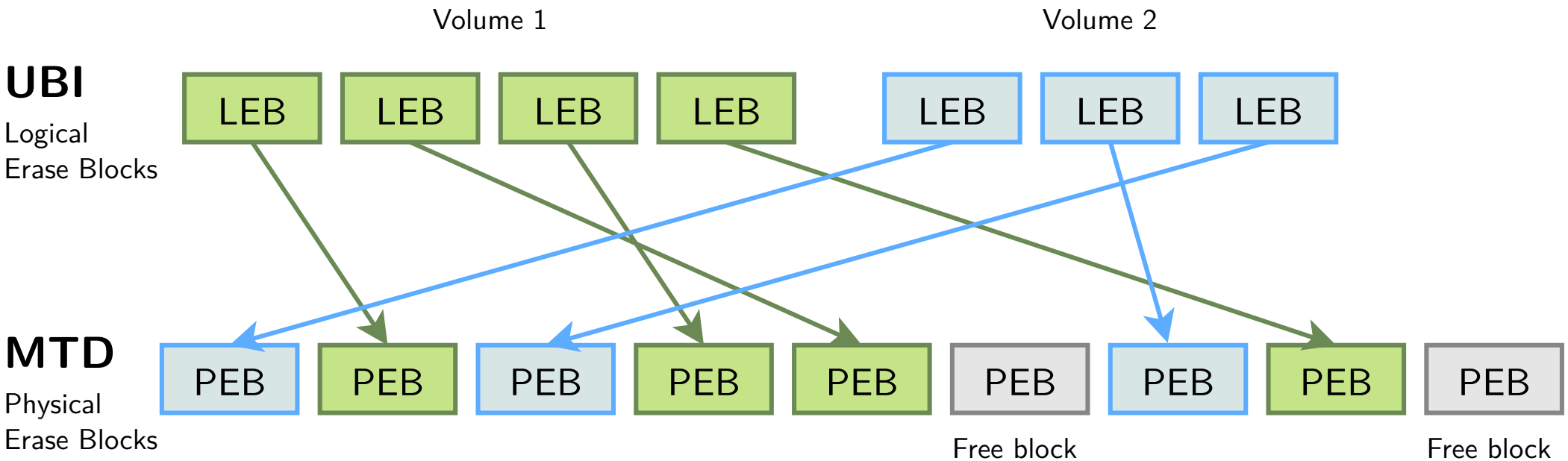
UBI: Unsorted Block Images

- Design choices:
  - Split the wear leveling and filesystem layers
  - Add some flexibility
  - Focus on scalability, performance and reliability
- Drawback: introduces noticeable space overhead, especially when used on small devices or partitions. JFFS2 still makes sense on small MTD partitions.
- Implements logical volumes on top of MTD devices (like LVM for block devices)
- Allows wear leveling to operate on the whole storage, not only on individual partitions.

http://www.linux-mtd.infradead.org/doc/ubi.html



UBI (2)



When there is too much activity on an LEB, UBI can decide to move it to another PEB with a lower erase count. Even read-only volumes participate to wear leveling!

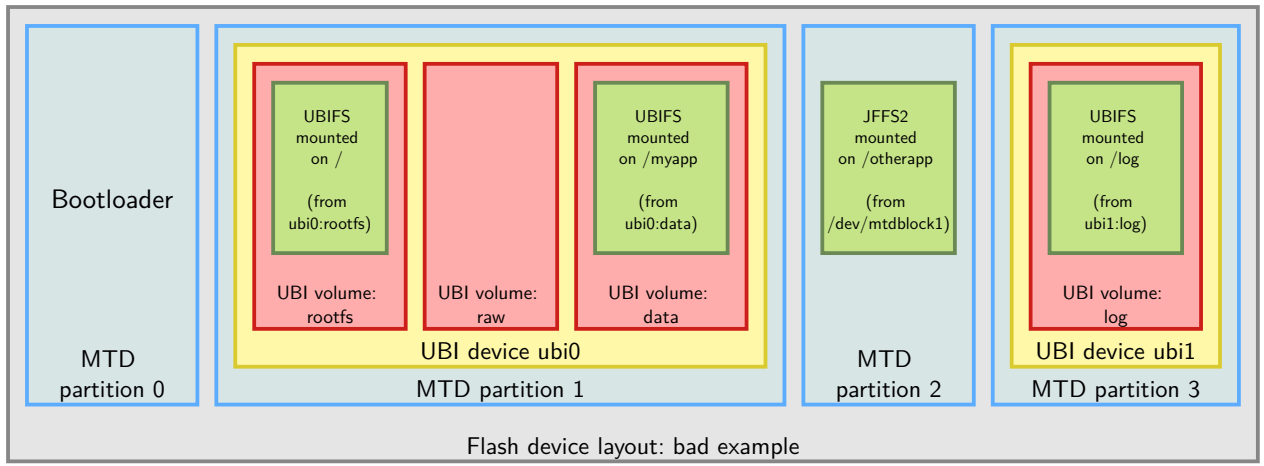


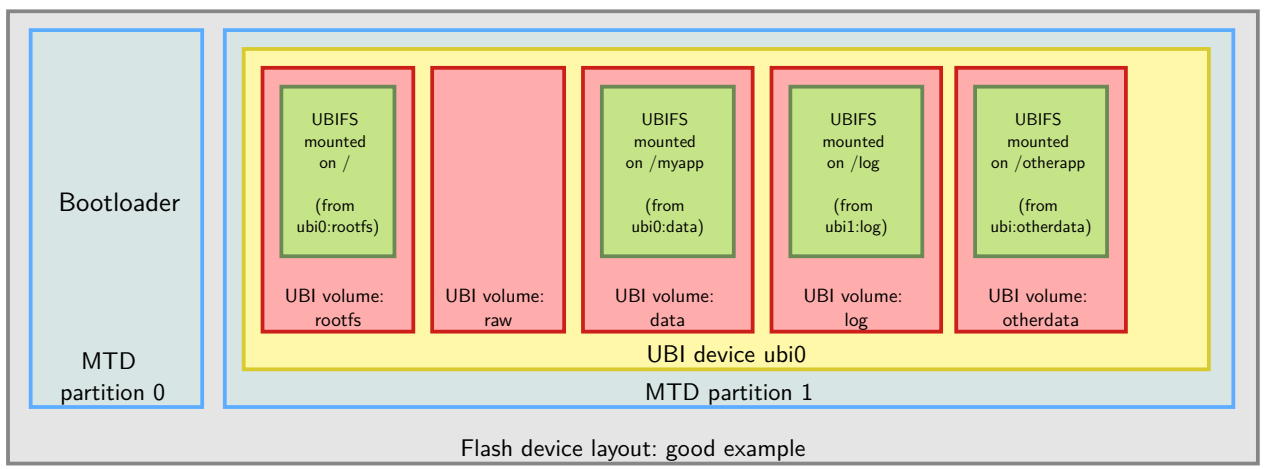
UBI: good practice

- UBI distributes erases all over the flash device: the more space you assign to a partition attached to the UBI layer the more efficient wear leveling will be.
- If you need partitioning, use UBI volumes, not MTD partitions.
- Some partitions will still have to be MTD partitions: e.g. the bootloaders.
- U-Boot now even supports storing its environment in a UBI volume!
- If you do need extra MTD partitions, try to group them at the beginning of the flash device (often more reliable area).



UBI: bad and good practice





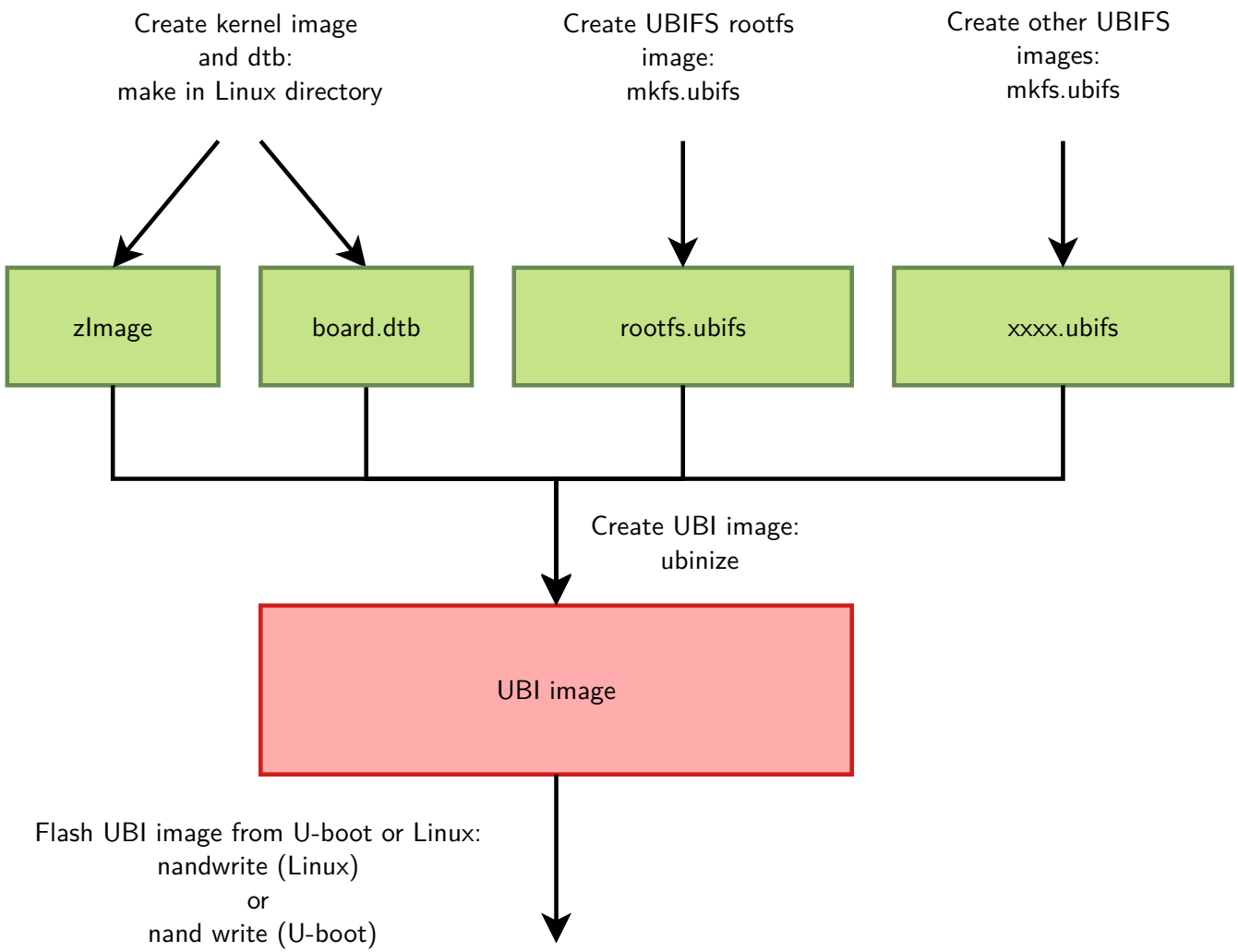


UBIFS *Unsorted Block Images File System*

- http://www.linux-mtd.infradead.org/doc/ubifs.html
- Journaling file system providing better performance than its predecessor (JFFS2) and addressing its scalability issues
- Can be mounted as the root filesystem too
- UBIFS filesystem images can be created using `mkfs.ubifs` from *mtd-utils*
- This image can then be flashed on a volume or included in a UBI image (`ubinize` command).



ubinize for UBI image creation





Linux: Block emulation layers

- Sometimes needed to use read-only block filesystems such as Squashfs and EROFS
- Linux provides two block emulation layers:
  - `mtdblock` (CONFIG_MTD_BLOCK): block devices emulated on top of MTD devices.
    - Named `/dev/mtdblockX`, one for each partition.
    - Originally the `mount` command wanted a block device to mount JFFS2 and YAFFS2.
    - don't write to `mtdblock` devices: bad blocks are not handled!
  - `ubiblock` (CONFIG_MTD_UBI_BLOCK): **read-only** block devices emulated on top of UBI volumes
    - Used on static (read-only) volumes
    - Usually named `/dev/ubiblockX_Y`, where X is the UBI device id and Y is the UBI volume id (example: `/dev/ubiblock0_3`)